CKAD Exam: The Complete Study Guide

The Certified Kubernetes Application Developer (CKAD) exam tests your ability to design, build, configure, and expose cloud-native applications for Kubernetes. This **performance-based exam** requires hands-on command-line work in a live cluster environment—memorizing theory alone won't suffice. You have **2 hours** to complete **15-20 tasks** and need **66%** to pass. The exam currently runs on **Kubernetes v1.34** and includes one free retake plus two Killer.sh simulator sessions.

This guide covers every exam objective with kubectl commands, YAML manifests, and practical tips organized by the five weighted domains.

Exam overview and environment

Attribute	Details
Duration	2 hours
Passing Score	66%
Format	15-20 performance-based tasks (not multiple choice)
Kubernetes Version	v1.34
Cost	$445 USD (includes 1 retake + 2 Killer.sh sessions)
Certification Validity	2 years

Domain weightages

Domain	Weight
Application Environment, Configuration and Security	**25%**
Application Design and Build	20%
Application Deployment	20%
Services and Networking	20%
Application Observability and Maintenance	15%

Allowed resources during the exam

You can access these documentation sites in a browser tab within the exam VM:

- https://kubernetes.io/docs/ (including search)
- https://kubernetes.io/blog/
- https://github.com/kubernetes/
- https://helm.sh/docs/

The exam environment includes `kubectl` with `k` alias and bash autocompletion pre-configured, plus `yq`, `curl`, `wget`, and `vim`.

Critical keyboard shortcuts

Action	Keys
Copy in terminal	Ctrl+Shift+C
Paste in terminal	Ctrl+Shift+V
Avoid (closes browser tabs)	Ctrl+W
Use instead	Ctrl+Alt+W

Domain 1: Application Environment, Configuration and Security (25%)

This largest-weighted domain covers ConfigMaps, Secrets, resource management, security contexts, ServiceAccounts, and Pod Security Standards.

ConfigMaps

ConfigMaps decouple configuration from container images, storing non-confidential key-value pairs with a **1 MiB size limit**. Spacelift

Imperative creation commands:

```bash
# From literals
kubectl create configmap my-config --from-literal=key1=value1 --from-literal=key2=value2

# From file (filename becomes key)
kubectl create configmap my-config --from-file=config.properties

# From file with custom key
kubectl create configmap my-config --from-file=custom-key=config.properties

# From directory (all files become keys)
kubectl create configmap my-config --from-file=./config-dir/

# From env-file format
kubectl create configmap my-config --from-env-file=app.env
```

Using ConfigMap as environment variables:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-env-pod
spec:
  containers:
  - name: app
    image: nginx
    env:
    - name: DATABASE_URL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: database_url
    # Or import all keys with prefix
    envFrom:
    - configMapRef:
        name: app-config
      prefix: APP_
```

Mounting ConfigMap as volume:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-volume-pod
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - name: config-volume
      mountPath: /etc/config
      readOnly: true
  volumes:
  - name: config-volume
    configMap:
      name: app-config
      items:                    # Optional: select specific keys
      - key: app.properties
        path: application.conf  # Custom filename
```

Documentation: https://kubernetes.io/docs/concepts/configuration/configmap/

Secrets

Secrets store sensitive data like passwords and tokens. Data is **base64-encoded** (not encrypted by default).

Secret types:

Type	Usage
`Opaque`	Default—arbitrary user data
`kubernetes.io/dockerconfigjson`	Docker registry credentials
`kubernetes.io/tls`	TLS certificate and key
`kubernetes.io/basic-auth`	Basic authentication

Imperative creation commands:

```bash
# Generic secret from literals
kubectl create secret generic my-secret \
  --from-literal=username=admin \
  --from-literal=password=secretpass

# Docker registry secret
kubectl create secret docker-registry my-registry-secret \
  --docker-server=registry.example.com \
  --docker-username=user \
  --docker-password=pass

# TLS secret
kubectl create secret tls my-tls-secret \
  --cert=tls.crt --key=tls.key

# Decode secret value
kubectl get secret my-secret -o jsonpath='{.data.password}' | base64 -d
```

Secret YAML with stringData (auto-encodes):

yaml

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:          # Plain text—automatically base64 encoded
  username: admin
  password: secret123
```

Using Secrets in Pods:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
spec:
  containers:
  - name: app
    image: nginx
    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: password
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: app-secret
      defaultMode: 0400
  imagePullSecrets:          # For private registries
  - name: my-registry-secret
```

Documentation: https://kubernetes.io/docs/concepts/configuration/secret/

Resource requirements and limits

Concept	Description
Requests	Minimum guaranteed resources (used for scheduling)
Limits	Maximum resources allowed
CPU	Compressible—throttled when exceeded (1 CPU = 1000m)
Memory	Non-compressible—container OOMKilled if exceeded

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: resource-demo
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "128Mi"
        cpu: "500m"
```

ResourceQuota (namespace-level limits):

yaml

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: mem-cpu-quota
spec:
  hard:
    requests.cpu: "2"
    requests.memory: "4Gi"
    limits.cpu: "4"
    limits.memory: "8Gi"
    pods: "10"
```

LimitRange (defaults and constraints):

yaml

```yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: limit-range
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: "512Mi"
    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
    max:
      cpu: "2"
      memory: "2Gi"
    min:
      cpu: "50m"
      memory: "64Mi"
```

Documentation: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/

SecurityContext

SecurityContext defines privilege and access control settings at Pod or Container level.

Complete secure Pod example:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: secured-pod
spec:
  securityContext:            # Pod-level
    runAsUser: 1000
    runAsGroup: 3000
    fsGroup: 2000
    runAsNonRoot: true
  containers:
  - name: app
    image: nginx
    securityContext:          # Container-level (overrides Pod)
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop:
        - ALL
        add:
        - NET_BIND_SERVICE
    volumeMounts:
    - name: tmp
      mountPath: /tmp
  volumes:
  - name: tmp
    emptyDir: {}
```

Key settings:

Setting	Description
`runAsUser`	UID to run container process
`runAsGroup`	Primary GID for processes
`fsGroup`	GID for volume ownership
`runAsNonRoot`	Fail if trying to run as root
`readOnlyRootFilesystem`	Mount root filesystem read-only
`allowPrivilegeEscalation`	Prevent gaining more privileges
`capabilities`	Add/drop Linux capabilities

Documentation: https://kubernetes.io/docs/tasks/configure-pod-container/security-context/

ServiceAccounts

ServiceAccounts provide identity for processes in Pods. Since K8s v1.22+, tokens are short-lived and auto-rotated.

```bash
# Create ServiceAccount
kubectl create serviceaccount my-sa

# Create short-lived token
kubectl create token my-sa --duration=2h
```

Using ServiceAccount in Pod:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-sa
spec:
  serviceAccountName: my-service-account
  automountServiceAccountToken: true    # Default; set false if not needed
  containers:
  - name: app
    image: nginx
```

Documentation: https://kubernetes.io/docs/concepts/security/service-accounts/

Pod Security Standards

Pod Security Admission enforces security standards via namespace labels.

Level	Description
`privileged`	Unrestricted—allows known privilege escalations
`baseline`	Minimally restrictive—prevents known escalations
`restricted`	Highly restrictive—hardening best practices

Label namespace for enforcement:

```bash
kubectl label namespace my-ns \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted
```

Documentation: https://kubernetes.io/docs/concepts/security/pod-security-standards/

Domain 2: Application Design and Build (20%)

This domain covers container images, Jobs/CronJobs, multi-container Pod patterns, and persistent storage.

Jobs and CronJobs

Jobs run Pods to completion for batch workloads. `kubernetes` Key parameters:

Field	Description	Default
`completions`	Successful completions needed	1
`parallelism`	Concurrent pods	1
`backoffLimit`	Retries before failure	6
`activeDeadlineSeconds`	Maximum runtime	None
`ttlSecondsAfterFinished`	Auto-cleanup delay	None

Imperative Job creation:

```bash
kubectl create job my-job --image=busybox -- echo "Hello World"

# Test CronJob immediately
kubectl create job test-job --from=cronjob/my-cronjob
```

Job YAML:

yaml

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: parallel-job
spec:
  completions: 5
  parallelism: 2
  backoffLimit: 4
  ttlSecondsAfterFinished: 100
  template:
    spec:
      containers:
      - name: worker
        image: busybox
        command: ["echo", "Processing"]
      restartPolicy: Never      # Required: Never or OnFailure
```

CronJob with schedule:

yaml

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-cron
spec:
  schedule: "0 2 * * *"             # Daily at 2am
  concurrencyPolicy: Forbid         # Allow|Forbid|Replace
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: backup-tool:latest
          restartPolicy: OnFailure
```

Cron schedule format: `minute hour day-of-month month day-of-week`

- Every 5 minutes: `*/5 * * * *`
- Daily at midnight: `0 0 * * *`
- Every Monday at 3am: `0 3 * * 1`

Documentation: https://kubernetes.io/docs/concepts/workloads/controllers/job/

Multi-container Pod patterns

Containers in the same Pod share network namespace (localhost) and storage volumes.

Pattern	Purpose	Lifecycle
Init Container	Setup/prerequisites	Runs before main, exits
Sidecar	Extend/enhance main	Runs alongside main
Ambassador	Proxy outbound connections	Runs alongside
Adapter	Transform/normalize output	Runs alongside

Init Container example:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: init-demo
spec:
  initContainers:
  - name: init-myservice
    image: busybox
    command: ['sh', '-c', 'until nslookup myservice; do sleep 2; done']
  containers:
  - name: main-app
    image: nginx
```

Native Sidecar (K8s v1.29+):

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: sidecar-demo
spec:
  initContainers:
  - name: log-shipper
    image: fluent-bit
    restartPolicy: Always      # Makes it a sidecar
    volumeMounts:
    - name: logs
      mountPath: /var/log/app
  containers:
  - name: main-app
    image: nginx
    volumeMounts:
    - name: logs
      mountPath: /var/log/nginx
  volumes:
  - name: logs
    emptyDir: {}
```

Commands for multi-container Pods:

```bash
kubectl logs my-pod -c container-name
kubectl logs my-pod -c init-container-name
kubectl exec -it my-pod -c container-name -- /bin/sh
```

Documentation: https://kubernetes.io/docs/concepts/workloads/pods/init-containers/

Volumes and Persistent Storage

Volume Type	Persistence	Use Case
`emptyDir`	Pod lifetime	Scratch space, shared cache
`hostPath`	Node lifetime	Dev/testing only
`configMap` / `secret`	ConfigMap/Secret data	Configuration
`persistentVolumeClaim`	Beyond pod	Databases, stateful apps

PersistentVolumeClaim:

yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce          # RWO|ROX|RWX|RWOP
  resources:
    requests:
      storage: 10Gi
  storageClassName: standard  # Triggers dynamic provisioning
```

Using PVC in Pod:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pod-with-pvc
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - name: storage
      mountPath: /data
  volumes:
  - name: storage
    persistentVolumeClaim:
      claimName: my-pvc
```

Commands:

bash

```bash
kubectl get pv
kubectl get pvc
kubectl get storageclass
```

Documentation: https://kubernetes.io/docs/concepts/storage/persistent-volumes/

Domain 3: Application Deployment (20%)

This domain covers Deployments, rolling updates, deployment strategies, Helm, and Kustomize.

Deployments and Rolling Updates

```bash
# Create deployment
kubectl create deployment nginx --image=nginx:1.14 --replicas=3

# Update image
kubectl set image deployment/nginx nginx=nginx:1.16

# Check rollout status
kubectl rollout status deployment/nginx

# View history
kubectl rollout history deployment/nginx

# Rollback to previous
kubectl rollout undo deployment/nginx

# Rollback to specific revision
kubectl rollout undo deployment/nginx --to-revision=2

# Pause/resume rollout
kubectl rollout pause deployment/nginx
kubectl rollout resume deployment/nginx

# Restart (trigger new rollout)
kubectl rollout restart deployment/nginx

# Scale
kubectl scale deployment/nginx --replicas=5
```

Deployment with rolling update strategy:

yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%          # Max pods over desired
      maxUnavailable: 25%    # Max pods unavailable
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.14.2
        ports:
        - containerPort: 80
```

Documentation: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

Blue-Green and Canary deployments

Blue-Green: Run two identical environments, switch traffic instantly via Service selector.

bash

```bash
# Switch from blue to green
kubectl patch service myapp-service -p '{"spec":{"selector":{"version":"green"}}}'
```

Canary: Deploy new version to small subset, route percentage of traffic.

yaml

```yaml
# Stable: 9 replicas with app=myapp, track=stable
# Canary: 1 replica with app=myapp, track=canary
# Service selects app=myapp (matches both, 90/10 split)
```

Helm basics

bash

```bash
# Repository management
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm search repo nginx

# Install
helm install my-release bitnami/nginx
helm install my-release bitnami/nginx --set replicaCount=3
helm install my-release bitnami/nginx -f values.yaml

# View releases
helm list
helm status my-release
helm history my-release

# Upgrade
helm upgrade my-release bitnami/nginx --set replicaCount=5

# Rollback
helm rollback my-release 1

# Uninstall
helm uninstall my-release
```

Documentation: https://helm.sh/docs/intro/using_helm/

Kustomize

Kustomize customizes Kubernetes configurations without templates.

kustomization.yaml:

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
namespace: production
namePrefix: prod-
images:
- name: nginx
  newTag: "1.19.0"
replicas:
- name: my-deployment
  count: 5
```

Commands:

```bash
# Preview output
kubectl kustomize ./

# Apply
kubectl apply -k ./

# Apply overlay
kubectl apply -k ./overlays/production
```

Documentation: https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/

Domain 4: Services and Networking (20%)

This domain covers Service types, Ingress, NetworkPolicies, and DNS.

Service types

Type	Description	Access
ClusterIP	Internal cluster IP (default)	Within cluster only
NodePort	Static port 30000-32767 on all nodes	`<NodeIP>:<NodePort>`
LoadBalancer	External load balancer (cloud)	External LB IP
ExternalName	CNAME to external DNS	Internal → external
Headless	`clusterIP: None`—DNS returns Pod IPs	Direct Pod access

```bash
# Expose deployment
kubectl expose deployment nginx --port=80 --target-port=8080

# NodePort
kubectl expose deployment nginx --port=80 --type=NodePort

# Generate YAML
kubectl expose deployment nginx --port=80 --type=NodePort --dry-run=client -o yaml
```

Service YAML:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - port: 80          # Service port
    targetPort: 8080    # Pod port
    nodePort: 30007     # Optional—auto-assigned if omitted
```

Documentation: https://kubernetes.io/docs/concepts/services-networking/service/

Ingress

Ingress exposes HTTP/HTTPS routes from outside the cluster. Requires an **Ingress Controller**.

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  ingressClassName: nginx      # Required
  tls:
  - hosts:
    - myapp.example.com
    secretName: tls-secret
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

Path types: `Exact` (exact match), `Prefix` (prefix match), `ImplementationSpecific`

Documentation: https://kubernetes.io/docs/concepts/services-networking/ingress/

NetworkPolicies

NetworkPolicies control Pod-to-Pod traffic at L3/L4. Requires CNI plugin support (Calico, Cilium).

Default deny all ingress:

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
spec:
  podSelector: {}      # Selects ALL pods
  policyTypes:
  - Ingress
```

Allow from specific pods:

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    - namespaceSelector:
        matchLabels:
          name: production
    ports:
    - protocol: TCP
      port: 8080
```

Allow egress with DNS:

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress
spec:
  podSelector:
    matchLabels:
      app: myapp
  policyTypes:
  - Egress
  egress:
  - to:
    - ipBlock:
        cidr: 10.0.0.0/24
    ports:
    - protocol: TCP
      port: 443
  - ports:                  # Allow DNS
    - protocol: UDP
      port: 53
```

Documentation: https://kubernetes.io/docs/concepts/services-networking/network-policies/

DNS for Services

Service DNS format: `<service>.<namespace>.svc.cluster.local`

Cross-namespace access:

bash

```bash
# From same namespace
curl my-service

# From different namespace
curl my-service.other-namespace
curl my-service.other-namespace.svc.cluster.local
```

Documentation: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/

Domain 5: Application Observability and Maintenance (15%)

This domain covers logging, probes, debugging, and API deprecations.

Logging

```bash
# Basic logs
kubectl logs nginx

# Specific container in multi-container pod
kubectl logs nginx -c sidecar

# Previous container (after restart/crash)
kubectl logs nginx --previous

# Stream logs
kubectl logs -f nginx

# Last 50 lines
kubectl logs --tail=50 nginx

# Since duration
kubectl logs --since=1h nginx

# All containers, all pods by label
kubectl logs -l app=nginx --all-containers=true
```

Documentation: https://kubernetes.io/docs/concepts/cluster-administration/logging/

Probes

Probe	Purpose	Action on Failure
Liveness	Is container running?	Restart container
Readiness	Can container accept traffic?	Remove from endpoints
Startup	Has app started?	Block other probes; restart on failure

Probe parameters:

Parameter	Default	Description
initialDelaySeconds	0	Wait before first probe
periodSeconds	10	Probe frequency
timeoutSeconds	1	Probe timeout
successThreshold	1	Successes to mark healthy
failureThreshold	3	Failures before action

All probe types:

yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: probe-demo
spec:
  containers:
  - name: app
    image: nginx
    ports:
    - containerPort: 8080
    startupProbe:
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 15
      periodSeconds: 10
    readinessProbe:
      tcpSocket:
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
    # Exec probe alternative:
    # livenessProbe:
    #   exec:
    #     command: ["cat", "/tmp/healthy"]
```

Documentation: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/

Debugging and troubleshooting

```bash
# Get pod status
kubectl get pods -o wide

# Describe for events and conditions
kubectl describe pod nginx

# Get events
kubectl get events --field-selector involvedObject.name=nginx
kubectl events --for pod/nginx

# Exec into container
kubectl exec -it nginx -- /bin/sh

# Port forward for testing
kubectl port-forward pod/nginx 8080:80
kubectl port-forward svc/nginx-service 8080:80

# Resource usage
kubectl top pods
kubectl top nodes

# Debug with ephemeral container
kubectl debug -it nginx --image=busybox
```

Common issues:

State	Cause	Troubleshooting
`Pending`	Scheduling issues	Check resources, node selectors, taints
`ImagePullBackOff`	Registry issues	Verify image name, check secrets
`CrashLoopBackOff`	App crashes	Check logs, `--previous` flag

API deprecations

```bash
# List API versions
kubectl api-versions

# List resources with API groups
kubectl api-resources

# Explain resource
kubectl explain deployment.spec.strategy
```

Documentation: https://kubernetes.io/docs/reference/using-api/deprecation-guide/

Quick reference cheat sheet

Essential imperative commands

```bash
# Pods
kubectl run nginx --image=nginx --port=80 --dry-run=client -o yaml

# Deployments
kubectl create deployment nginx --image=nginx --replicas=3

# Services
kubectl expose deployment nginx --port=80 --type=NodePort

# ConfigMaps
kubectl create configmap my-cm --from-literal=key=value

# Secrets
kubectl create secret generic my-secret --from-literal=pass=secret

# Jobs
kubectl create job my-job --image=busybox -- echo "hello"

# CronJobs
kubectl create cronjob my-cron --image=busybox --schedule="*/5 * * * *" -- date

# ServiceAccounts
kubectl create serviceaccount my-sa
```

Useful flags

Flag	Usage
`--dry-run=client -o yaml`	Generate YAML without creating
`-o jsonpath='{.data.key}'`	Extract specific field
`-w` or `--watch`	Watch for changes
`-l app=nginx`	Filter by label
`--all-namespaces` or `-A`	All namespaces

YAML generation shortcuts

```bash
# Generate and save
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deploy.yaml

# Edit in place
kubectl edit deployment nginx

# Apply changes
kubectl apply -f deploy.yaml
```

Exam day tips

Time management: With 15-20 tasks in 120 minutes, aim for **6-8 minutes per task**. Flag difficult questions and return later.

Use imperative commands: `kubectl create` and `kubectl run` are faster than writing YAML from scratch. Use `--dry-run=client -o yaml` to generate base YAML, then modify.

Leverage documentation: Bookmark key pages in kubernetes.io. The search function is allowed—use it to find YAML examples quickly.

Context switching: Pay attention to the cluster context specified for each question. Use `kubectl config use-context` as instructed.

Validation: After creating resources, verify with `kubectl get` and `kubectl describe`. Check that pods are Running and endpoints exist.

Practice environment: The Killer.sh simulator sessions included with your exam registration closely match the real exam—complete both before test day.

Official documentation reference

Topic	URL
ConfigMaps	https://kubernetes.io/docs/concepts/configuration/configmap/
Secrets	https://kubernetes.io/docs/concepts/configuration/secret/
Resource Management	https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
SecurityContext	https://kubernetes.io/docs/tasks/configure-pod-container/security-context/
ServiceAccounts	https://kubernetes.io/docs/concepts/security/service-accounts/
Pod Security Standards	https://kubernetes.io/docs/concepts/security/pod-security-standards/
Jobs	https://kubernetes.io/docs/concepts/workloads/controllers/job/
CronJobs	https://kubernetes.io/docs/concepts/workloads/controllers/cron-jobs/
Init Containers	https://kubernetes.io/docs/concepts/workloads/pods/init-containers/
Sidecar Containers	https://kubernetes.io/docs/concepts/workloads/pods/sidecar-containers/
Volumes	https://kubernetes.io/docs/concepts/storage/volumes/
Persistent Volumes	https://kubernetes.io/docs/concepts/storage/persistent-volumes/
Deployments	https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
Kustomize	https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/
Services	https://kubernetes.io/docs/concepts/services-networking/service/
Ingress	https://kubernetes.io/docs/concepts/services-networking/ingress/
NetworkPolicies	https://kubernetes.io/docs/concepts/services-networking/network-policies/
DNS	https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/
Probes	https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
Logging	https://kubernetes.io/docs/concepts/cluster-administration/logging/
Debugging	https://kubernetes.io/docs/tasks/debug/debug-application/
API Deprecations	https://kubernetes.io/docs/reference/using-api/deprecation-guide/
Helm Docs	https://helm.sh/docs/
CKAD Curriculum	https://github.com/cncf/curriculum
Exam Handbook	https://docs.linuxfoundation.org/tc-docs/certification/lf-handbook2

Conclusion

Success in the CKAD exam comes from hands-on practice, not memorization. The exam tests practical skills—creating, modifying, and troubleshooting Kubernetes applications under time pressure. Focus your preparation on the **25% Configuration/Security domain** and **20% Services/Networking domain**, which together comprise nearly half the exam weight.

Build muscle memory for imperative kubectl commands and learn to navigate kubernetes.io documentation efficiently. Complete both Killer.sh simulator sessions, as they closely mirror exam difficulty and interface. Time management is critical: move quickly through tasks you know, flag uncertain ones for review, and always verify your work with `kubectl get` and `kubectl describe` before moving on.